

EXPERIMENT - 34

1. Problem Statement

Write a C program to perform the following menu-driven operations using separate functions:

- Read three variable-length 2D arrays and verify the left distributive property: $A*(B+C) = (A*B) + (A*C)$. The dimensions of the input matrices are A ($m \times n$), B ($n \times p$), C ($n \times p$).
- Read three variable-length 2D arrays and verify the right distributive property: $(A+B)*C = (A*C) + (B*C)$. The dimensions of the input matrices are A ($m \times n$), B ($m \times n$), C ($n \times p$).
- Exit.

The matrices obtained on both the left-hand side and the right-hand side of each distributive property have dimensions ($m \times p$). Both resulting matrices are displayed and the program states whether the selected distributive property is verified.

2. Algorithm

Main Program

Step 1: START

Step 2: Display the menu and read choice.

Step 3: Using a switch statement on choice:

 If choice is 1, call leftDistributive().

 If choice is 2, call rightDistributive().

 If choice is 3, display "Exiting the program."

Step 4: Repeat from Step 2 while choice is not equal to 3.

Step 5: STOP

Function leftDistributive()

Step 1: Read m, n and p, and declare a of size $m \times n$, b and c of size $n \times p$, bc of size $n \times p$, and ab, ac, left and right of size $m \times p$.

Step 2: Read the elements of A, then of B, then of C.

Step 3: Find $bc = B + C$, then $left = A \times bc$.

Step 4: Find $ab = A \times B$ and $ac = A \times C$, then $right = ab + ac$.

Step 5: Display $A \times (B + C)$ and $(A \times B) + (A \times C)$.

Step 6: If the two are equal place by place, display that the property is verified.

Step 7: Return to the calling function.

Function rightDistributive()

Step 1: Read m, n and p, and declare a and b of size $m \times n$, c of size $n \times p$, ab of size $m \times n$, and ac, bc, left and right of size $m \times p$.

Step 2: Read the elements of A, then of B, then of C.

Step 3: Find $ab = A + B$, then $left = ab \times C$.

Step 4: Find $ac = A \times C$ and $bc = B \times C$, then $right = ac + bc$.

Step 5: Display $(A + B) \times C$ and $(A \times C) + (B \times C)$.

Step 6: If the two are equal place by place, display that the property is verified.

Step 7: Return to the calling function.

Functions `add()` and `multiply()`

Step 1: In `add()`, repeat for every i and j : set $z[i][j] = x[i][j] + y[i][j]$.

Step 2: In `multiply()`, repeat for $i = 0$ to $r1 - 1$ and for $j = 0$ to $c2 - 1$:

Set $z[i][j] = 0$.

Repeat for $k = 0$ to $c1 - 1$: add $x[i][k] \times y[k][j]$ to $z[i][j]$.

Step 3: Return to the calling function.

3. Test Case Table

#	Operation	Input	Expected Output	Actual Output	Result
1	1. Left	$m \ n \ p = 2 \ 3 \ 2$, $A = 1 \ 2 \ 3 / 4 \ 5 \ 6$, $B = 1 \ 2 / 3 \ 4 / 5 \ 6$, $C = 7 \ 8 / 9 \ 10 / 11 \ 12$	$A \times (B + C) = 80 \ 92 / 188 \ 218$	80 92 / 188 218	PASS
2	1. Left	the same three matrices	$(A \times B) + (A \times C)$ is the same 2×2 matrix	80 92 / 188 218	PASS
3	2. Right	$m \ n \ p = 2 \ 2 \ 3$, $A = 1 \ 2 / 3 \ 4$, $B = 5 \ 6 / 7 \ 8$, $C = 1 \ 0 \ 2 / 0 \ 1 \ 3$	$(A + B) \times C = 6 \ 8 \ 36 / 10 \ 12 \ 56$	6 8 36 / 10 12 56	PASS
4	2. Right	the same three matrices	$(A \times C) + (B \times C)$ is the same 2×3 matrix	6 8 36 / 10 12 56	PASS
5	2. Right	$m \ n \ p = 3 \ 2 \ 1$, so all three sizes differ	Both sides come out as the 3×1 matrix 46 / 66 / 86	46 / 66 / 86	PASS
6	1. Left	$m \ n \ p = 1 \ 1 \ 1$ with 5, 6 and 7	Both sides come out as 65	65	PASS
7	3. Exit	choice = 3	Exiting the program, and the loop ends	Loop ended	PASS

4. Source Code

distributive.c

```
/* Experiment 34. Multiplication distributes over addition from both
sides. Whichever way the three matrices are grouped, both sides
finish as an m x p matrix, and that is the only reason they can be
compared at all. The arrays are variable length, because the sizes
are only known at run time. */

#include <stdio.h>

void leftDistributive(void);
void rightDistributive(void);

/* The number of columns comes before the matrix, so that the compiler
knows the shape of each row. */
void readMatrix(int rows, int cols, int m[rows][cols], char name)
{
    int i, j;

    printf("Enter the elements of matrix %c (%d x %d):\n", name, rows, cols);
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            scanf("%d", &m[i][j]);
}

void showMatrix(int rows, int cols, int m[rows][cols], const char *label)
{
    int i, j;

    printf("%s\n", label);
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            if (j > 0)
                printf("\t");
            printf("%d", m[i][j]);
        }
        printf("\n");
    }
}

/* Plain place by place addition, so both matrices have to be the same
size. */
void add(int rows, int cols, int x[rows][cols], int y[rows][cols],
        int z[rows][cols])
{
    int i, j;

    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            z[i][j] = x[i][j] + y[i][j];
}

/* z becomes x times y. c1 is the shared side, the columns of x and the
rows of y, and it is the one that disappears from the answer. */
void multiply(int r1, int c1, int c2, int x[r1][c1], int y[c1][c2],
            int z[r1][c2])
{
    int i, j, k;

    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++) {
            z[i][j] = 0;
            for (k = 0; k < c1; k++)
                z[i][j] += x[i][k] * y[k][j];
        }
}

int same(int rows, int cols, int x[rows][cols], int y[rows][cols])
{
}
```

```

    int i, j;

    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            if (x[i][j] != y[i][j])
                return 0;
    return 1;
}

void verdict(int rows, int cols, int x[rows][cols], int y[rows][cols])
{
    if (same(rows, cols, x, y))
        printf("Both sides are equal, so the distributive property is verified\n");
    else
        printf("The two sides are not equal\n");
}

int main(void)
{
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Verify the left distributive property\n");
        printf("2. Verify the right distributive property\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: leftDistributive(); break;
            case 2: rightDistributive(); break;
            case 3: printf("Exiting the program.\n"); break;
        }
    } while (choice != 3);

    return 0;
}

/* B and C are both n x p here, so they are the pair that gets added,
   and A multiplies the sum from the left. */
void leftDistributive(void)
{
    int m, n, p;

    printf("Enter the values of m, n and p: ");
    scanf("%d %d %d", &m, &n, &p);

    int a[m][n], b[n][p], c[n][p];
    int bc[n][p], ab[m][p], ac[m][p], left[m][p], right[m][p];

    readMatrix(m, n, a, 'A');
    readMatrix(n, p, b, 'B');
    readMatrix(n, p, c, 'C');

    add(n, p, b, c, bc);
    multiply(m, n, p, a, bc, left);

    multiply(m, n, p, a, b, ab);
    multiply(m, n, p, a, c, ac);
    add(m, p, ab, ac, right);

    showMatrix(m, p, left, "A x (B + C) =");
    showMatrix(m, p, right, "(A x B) + (A x C) =");
    verdict(m, p, left, right);
}

/* This is the one place where the two functions really differ. Here A
   and B are both m x n, so ab comes out as m x n and not m x p like in
   the other function, and C multiplies from the right. I checked that
   line twice, because it is easy to copy the wrong size across. */
void rightDistributive(void)
{
    int m, n, p;

```

```

printf("Enter the values of m, n and p: ");
scanf("%d %d %d", &m, &n, &p);

int a[m][n], b[m][n], c[n][p];
int ab[m][n], ac[m][p], bc[m][p], left[m][p], right[m][p];

readMatrix(m, n, a, 'A');
readMatrix(m, n, b, 'B');
readMatrix(n, p, c, 'C');

add(m, n, a, b, ab);
multiply(m, n, p, ab, c, left);

multiply(m, n, p, a, c, ac);
multiply(m, n, p, b, c, bc);
add(m, p, ac, bc, right);

showMatrix(m, p, left, "(A + B) x C =");
showMatrix(m, p, right, "(A x C) + (B x C) =");
verdict(m, p, left, right);
}

```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra distributive.c -o distributive` so that even the small warnings would show up, and it compiled clean in the first attempt itself.

Left Distributive Property, $m \ n \ p = 2 \ 3 \ 2$

```

adilshakil — -zsh — 123x33
Last login: Sun Sep 27 09:41:18 on ttys000
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra distributive.c -o distributive
adilshakil@ADILs-MacBook-Air MyAssignment % ./distributive

MENU
1. Verify the left distributive property
2. Verify the right distributive property
3. Exit
Enter your choice: 1
Enter the values of m, n and p: 2 3 2
Enter the elements of matrix A (2 x 3):
1 2 3
4 5 6
Enter the elements of matrix B (3 x 2):
1 2
3 4
5 6
Enter the elements of matrix C (3 x 2):
7 8
9 10
11 12
A x (B + C) =
80      92
188     218
(A x B) + (A x C) =
80      92
188     218
Both sides are equal, so the distributive property is verified

```

Right Distributive Property, m n p = 2 2 3

```
adilshakil --zsh-- 123x33

MENU
1. Verify the left distributive property
2. Verify the right distributive property
3. Exit
Enter your choice: 2
Enter the values of m, n and p: 2 2 3
Enter the elements of matrix A (2 x 2):
1 2
3 4
Enter the elements of matrix B (2 x 2):
5 6
7 8
Enter the elements of matrix C (2 x 3):
1 0 2
0 1 3
(A + B) x C =
6      8      36
10     12     56
(A x C) + (B x C) =
6      8      36
10     12     56
Both sides are equal, so the distributive property is verified
```

Right Distributive Property with All Three Sizes Different, and Exit

```
adilshakil --zsh-- 123x33

1. Verify the left distributive property
2. Verify the right distributive property
3. Exit
Enter your choice: 2
Enter the values of m, n and p: 3 2 1
Enter the elements of matrix A (3 x 2):
1 2
3 4
5 6
Enter the elements of matrix B (3 x 2):
7 8
9 10
11 12
Enter the elements of matrix C (2 x 1):
2
3
(A + B) x C =
46
66
86
(A x C) + (B x C) =
46
66
86
Both sides are equal, so the distributive property is verified

MENU
1. Verify the left distributive property
2. Verify the right distributive property
3. Exit
Enter your choice: 3
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
```

All seven test cases came out right, and I worked the first one out by hand too. $B + C$ is 8 10 / 12 14 / 16 18, and A times that gives 80 92 / 188 218. Coming the other way, $A \times B$ is 22 28 / 49 64 and $A \times C$ is 58 64 / 139

154, and those two add up to exactly the same matrix.

The two options look almost the same on paper but the sizes are not, and that is the bit I had to keep straight. In the left one B and C are both $n \times p$, so they are the pair that gets added. In the right one A and B are both $m \times n$, so $A + B$ is $m \times n$ and not $m \times p$, which is easy to get wrong if the line is copied across from the other function. Either way both sides land on $m \times p$, so they can be compared. I ran the right one with $m \ n \ p = 3 \ 2 \ 1$, where no two sizes are equal, just to be sure nothing was quietly depending on that.